07

Event Recommendation System

In this chapter, we design an event recommendation system similar to Eventbrite.it. Eventbrite.it is a popular event management and ticketing marketplace where people can create, promote, and manage events. It accommodates various professional, recreational, and social gatherings, ensuring attendees connect to events.

Clarifying Requirements

Here's a conversation between a candidate and an interviewer.

Candidate: What is the business objective? Can I assume the main business objective is to increase ticket sales?
Interviewer: Yes, that sounds good.
Candidate: Besides attending an event, can users book tickets or restaurants on the platform?
Interviewer: For simplicity, let's assume only events are supported.
Candidate: Is an event considered an upcoming event if the event has not yet begun, or is it considered an upcoming event if the event has not ended?
Interviewer: That's an excellent question!
Candidate: Can users book tickets on behalf of other users? I ask because we have access to the location of the event, price range, and the number of remaining tickets.
Interviewer: Sure, those are fair assumptions.
Candidate: Do we have any sponsored events?
Interviewer: We don't have a hard-labeled dataset. You can start short and your interaction data to construct the training dataset.
Candidate: Do we have any data on the user's social connections?
Interviewer: Yes, we have access to the user's friends and their attendance data. Social connections usually are quite useful in event recommendation systems.
Candidate: Can users become friends on the platform? Friendship information is valuable for building a personalized event recommendation system.
Interviewer: Unfortunately, no. Yes, let's assume users can form friendships on our platform. In Eventbrite.it International, meaning if a user's friend attends an event, it's more likely that the user will attend the event.
Candidate: Can users invite others to events?
Interviewer: Yes.
Candidate: Can a user RSVP to an event?
Interviewer: For simplicity, let's assume only a registration option is available for an event.
Candidate: Are the events free or paid?
Interviewer: Both are supported.
Candidate: Are there any content bans on events?
Interviewer: We have around 1 million local events every month.
Candidate: Do we have any hard-labeled data for training a model?
Interviewer: Anyone can have one million unique users per day.

The main objective of this system is to increase ticket sales by recommending events to users. It's important to calculate the distance and price range between the user and the event efficiently. Can we assume that we have access to a fast Google Maps API or other map services that can be used to obtain such data?

Interviewer: Good point. Assume we can use Google Maps services to obtain location data.

Let's summarize the problem statement. We are tasked with building a recommendation system, which displays a personalized feed of events for users to attend. The system should support various interactions, including viewing event details, saving events, inviting other users to attend, and their friendships. The training data should be constructed rather than user interactions. The primary goal of the system is to increase total ticket sales.

Frame the problem as an ML Task

Defining the ML objective

Based on the requirements, the business objective is to increase ticket sales. One way to translate this into a well-defined ML objective is to maximize the number of event registrations.

Specifying the system's input and output

The input to the system is a user and the output is the top n events ranked by relevance to the user.

Choosing the right ML category

There are different ways to address a recommendation problem:

- Simple rules, such as popularity ranking
- Embedding based approach such as content-based or collaborative filtering
- Deep learning or ranking models

Rule-based methods are good starting points because they are quick to implement. However, ML-based approaches usually lead to better outcomes. In this chapter, we will implement the two most popular approaches: content-based filtering and collaborative filtering.

CF is a class of algorithms/techniques that apply supervised machine learning to solve ranking problems. The ranking problem can be solved using a binary classification model, where the input is a user and an item, and the output is the probability that the user will interact in the future? There are generally three CF approaches: pointwise, pairwise, and listwise. Let's briefly examine each. Note that a detailed explanation of these approaches is beyond the scope of this book. If you're interested in learning more about ranking, consider reading Learning to Rank.

Pointwise CF

In this approach, we go over each item and predict the relevance between the user and the item, using classification or regression methods. Note that the output of the model is a probabilistic interpretation of the relevance.

The final ranking is achieved by sorting the predicted relevance scores.

Pairwise CF

In this approach, we go through each item twice and predicts which item is more relevant to the query.

Some of the most popular pairwise CF algorithms are RankNet [2], LambdaRank [3], and LambdaMART [4].

Listwise CF

Listwise approaches predict the optimal ordering of an entire list of items, given the query.

Some popular listwise CF algorithms are SoftRank [5], ListNet [6], and AdaRank [7].

In general, pairwise and listwise approaches produce more accurate results, but they are more difficult to implement and train. For simplicity, we will use a pointwise approach in this chapter. Note that we can still use a pairwise or listwise approach in the future if we struggle over a close and predictable baseline. Let's summarize the pros and cons of each CF approach.

Pointwise CF

- Pros: Simple to implement and train.
- Cons: Lower accuracy.

Pairwise CF

- Pros: Higher accuracy than pointwise approaches.
- Cons: Difficult to implement and train.

Listwise CF

- Pros: Higher accuracy than pairwise approaches.
- Cons: Difficult to implement and train.

Data Preparation

Data engineering

To engineer good features, we need first to understand the two main data sources available in the system. Since an event management platform is highly personal around users and events, we assume the following data are available:

- Users
- Events
- Friendships
- Interactions

Users

The user data schema is shown below:

ID	Gender	Birthdate	City	Country	Language	Time zone

Table 1: User data schema

Events

Table 2 shows what the event data might look like.

ID	Name	Description	Area	Location	Date/Time
1	Music Concert	Our jazz fest is filled with ...	200 square meters	American Airlines Arena Miami, FL	08/18/2022 7:00 - 10:00
2	Tech Talk	Learn about Meta's latest AI breakthroughs ...	100 square meters	Online Meta's HQ Menlo Park, CA	08/02/2022 7:00 - 10:00
3	Art Theatre	Art Theatre and Image Art Gallery ...	250 square meters	Chase Center 90 S. Ave Jose, CA	08/05/2022 19:00 - 22:30

Table 2: Event data

Friendships

In Table 3, each row represents a friendship formed between two users, along with the timestamp of when it was formed.

User ID 1	User ID 2	Timestamp when friendship was formed
1	20	1656800134
2	18	1656432123
15	20	1656123543

Table 3: Friendship data

Interactions

Table 4 shows user interaction data, such as event registrations, invitations, and impressions. In practice, we may store interaction data in different databases, but for simplicity, we include them in a single table.

User ID	Event ID	Interaction type	Interaction value	Location (lat, long)	Timestamp
2	18	Impression		38.8951 -77.0364	1656800134
4	10	Register	Confirmation number	38.8951 -77.0364	1656800134
4	10	Invite	User ID	40.6892 -74.0445	1656800134

Table 4: Interaction data

Feature engineering

Event based recommendations are time-dependent. For instance, if a user attended a music concert last Saturday, it's unlikely that the user will attend another music concert this Saturday. Therefore, it's crucial to incorporate the time component into the model. Note that we can easily transform raw features into time-based features such as the number of days between the event's creation and the event's start date, the time difference between the event's start and the user's average work scores of previous events, and the number of days since the event's last update.

Time-based features:

- Event's creation time
- Event's last update time
- Time difference between the event's start and the user's average work scores of previous events
- Time difference between the event's start and the user's last login time

Is the event in the same country and city as the user?

Some users may prefer events that are close to them. It is unlikely that a user in the same country and city attends they are located. The following are some of the relevant features:

- Is the event's country the same as the user's country, binary feature (1 or 0)
- If the user's city is the same as the event's city, the feature is 1, otherwise 0

Is the user comfortable with the distance?

Some users only prefer events that are very close to their location, while others prefer events that are further away. We can the following features to capture this:

- The distance between the user's location and the event's location. They value can be obtained from external APIs and bucketized into the categories. For example:
  - 0 less than a mile
  - 1 1-5 miles
  - 2 5-10 miles
  - 3 10-20 miles
  - 4 20-50 miles
  - 5 >100 miles
- Distance uniformity: Difference between the distance to an event and the average distance (in radio, the median or percentile range can be used) to events previously registered by the user.

Time-related features:

How convenient is the time remaining until an event?

Some users may plan events a few days in advance, while others don't. Let's create the following features to capture this:

- The remaining time until the event begins. The features can be bucketized into different categories and and then encoded. For example:
  - 0 less than 1 hour left until the event starts
  - 1 1-3 hours
  - 2 3-6 hours
  - 3 6-12 hours
  - 4 12-18 hours
  - 5 18-24 hours
  - 6 1-3 days
  - 7 3-7 days
  - 8 >7 days
- Remaining time uniformity: Difference between "remaining time" and average "remaining time" of events previously registered by the user.
- The estimated travel time from the user's location to the event's location. The value will be obtained from external services such as Google Maps API.
- Estimated travel time uniformity: The difference between the estimated travel time to the event in question and the average estimated travel time of events previously registered by the user.

Are the days and time convenient for the user?

Some users may prefer events that occur at weekends, while others prefer weekdays. Some users prefer events in the morning, while others prefer events in the evening. We can use the following features to capture this information:

- Is the event a weekend? Binary feature (1 or 0)
- Is the event's start date a weekend? If the event's start date is a weekend, the feature is 1, otherwise 0. Note that a user may attend an event on Monday, Tuesday, or Wednesday, but the event's start date is Saturday. In this case, the first two features will be useless, but the last feature may be helpful. For example, if a user attended an event on Monday, Wednesday, or Saturday, the event's start date is Saturday, and the user's recommendations for this user. The four user profiles can be created using a similar approach. Similarly, we will also use four similarity

A summary of time related features is shown in Figure 13.

Social-related features

How many people are attending this event?

In general, users are more likely to register for an event if there are a lot of other attendees. Let's extract the following features to capture this:

- The number of users registered for this event
- The ratio of the number of registered users to the number of attendees
- Registered user uniformity: The difference between the number of registered users for the event in question and previously registered events.

Features related to attendance by friends

Let's create the following features to capture if friends are attending or invited, and some of the features are user-specific:

- Number of friends attending and registered for the event
- The ratio of the number of registered friends to the total number of friends
- Registered friends uniformity: difference between the number of registered friends for the event in question and previously registered events.

Is the user invited to this event by others?

Users are more likely to attend events to which they are invited. Some features that might be helpful are:

- The number of friends who invited the user to the event
- The ratio of the number of friends who invited the user to the event

Is the event's host a friend of the user?

Users are more likely to attend events hosted by their friends. We create a binary feature to reflect this. If the event's host is the user's friend, the value is 1, otherwise, 0.

How often has the user attended previous events created by this host?

Some users are interested in following a particular host's events.

User-related features

Age and gender

Some events are geared toward specific ages and genders. For example, "Women in Tech" and "Life lessons to search in your 30 s" are more suitable for women in their 30s. Let's create the following features to capture this information:

- User's gender, one-hot encoded (or hash encoding)
- User's age, bucketized into multiple categories and encoded with one-hot encoding.

Event-related features

Price of event

The price of an event might affect the user's decision to register for it. Some features to use are:

- Event's price, bucketized into a few categories. For example:
  - 0 free
  - 1 0-49USD
  - 2 50-99USD
  - 3 100-199USD
  - 4 >200USD
- **Price uniformity**: Difference between the price of the event in question and the average price of events previously registered by the user.

How similar is this event's description to previously registered descriptions?

Text features can be very helpful in determining if a user will register for an event. For example, if the word "concert" repeatedly appears in the descriptions of previous events, it may indicate the user is interested in concert events. To capture this, we create a feature that represents the similarity between the event's description and the descriptions of previously registered events by the user. To calculate the similarity, we can bucketize the event's cosine similarity using TF-IDF, and encode it by one-hot encoding or other methods.

Note, Note features might be noisy, as descriptions are manually provided by hosts. We can experiment by trading our model with and without the text features.

Figure 14 shows an overview of user features, event features, and social related features.

Data Preprocessing

Most tabular datasets will be categorical. There are lots of other preprocessing techniques that can be created in practice. For example, we can bucketize numerical features, such as the number of friends, into categorical features, and create binary features. We can also apply log or sqrt transforms, or normalize features to reduce the skewness of the distribution. We can use it as a pointwise and then discretize feature values into a fixed number of buckets, and then treat the bucket index as a categorical feature.

Text features, such as the event's description and the user's description, are more complicated. These embedding vectors are used to represent the textual information. For simplicity, we will not use text features in this chapter.

Model Development

Model selection

Tabular classification problems can be solved by various ML methods. Let's take a look at the following:

- Logistic regression
- Decision tree
- Gradient boosted decision tree (GBDT)
- Neural network

Logistic regression and decision trees are very intuitive. Let's take a look at the following:

CF models the probability of a binary outcome by using a linear combination of one or multiple features. For the details of CF, refer to [1].

Let's see the pros and cons of LR:

Pros:

- **Fast inference speed**: Computing a weighted combination of input features is fast.
- **Low memory footprint**: Only the learned coefficients need to be stored in memory, which is not huge.
- Stable and robust due to low variance and high bias (Figure 17b).
- **Interpretable and easy to understand**: The weights assigned to each feature indicate the importance of different features, which gives an insight into why a decision was made.

Cons:

- **Poor linear problems can't be solved** (AND, XR, Gates 3 and 4 linear combination of input features).
- **Feature engineering is required**: Unlike tree-based models, LR can't learn complex feature interactions of an other a target variable and input variable (example is a linear combination of input features, XOR, XR).

In tree systems, the number of linear features can be very large. Often, these features have complex and non-linear relations with the target variable (binary classification). This complicates logistic regression (LR) and trees.

Decision tree

Decision trees offer another type of learning methods that can a few-like model of decisions and their possible consequences for make predictions. Figure 18 shows a simple decision tree with two features: age and gender. It also shows the corresponding decision boundary. Note that the decision boundary is perpendicular to the feature axis, because the decision tree only splits on one feature at a time.

Figure 115: Decision tree (left) and the learned decision boundary (right)

Pros:

- **Fast training**: Decision trees are quick to train.
- **Feature importance**: Decision trees make predictions quickly at inference time.
- **Scale-invariant**: Unlike many other algorithms, decision trees require no standardization or scaling, since the algorithm does not depend on the distribution of the input features.
- **Interpretable and easy to understand**. Visualizing the tree provides good insights into why a decision was made and what the important decision factors are.

Cons:

- **Non-optimal decision boundary**: Decision tree models produce decision boundaries that are parallel to the axes in the feature space (Figure 115). This may not be the optimal way to find a decision boundary for certain data distributions.
- **Overfitting**: Decision trees are very sensitive to small variations in data. A small change in input data may lead to different outcomes at serving time. Similarly, a small change in training data can lead to a totally different tree structure. This is a major issue and makes predictions less reliable.

In practice, naive decision trees are rarely used. The reason is that they are too sensitive to variations of input data. To reduce the sensitivity of decision trees, two techniques are commonly used:

- Bootstrap aggregation (bagging)
- Boosting

These two techniques are widely used across the tech industry. It's essential to understand how they work. Let's take a closer look.

Bagging

Bagging is the ensemble learning method that trains a set of IML models in parallel, on multiple subsets of the training data. In addition, bootstrapping is used to ensure the models are combined to make a final prediction. This significantly reduces the model's sensitivity to the changes in data (variance).

One example of bagging is the commonly used "random forest" model (Figure 116). Random forest builds multiple decision trees in parallel during training, to reduce the model's sensitivity. To make a prediction, each decision tree independently predicts the output class (positive or negative) of the given input, and then a voting mechanism is used to combine these predictions to make a final prediction. Figure 116 shows a random forest with three decision trees.

Figure 116: Random forest

The bagging technique has the following advantages:

- Reduces the effect of overfitting (high variance).
- Does not significantly increase training time because the decision trees can be trained in parallel.
- Does not add much latency at the inference time because decision trees can process the input in parallel.

Despite its advantages, bagging is not helpful when the model faces underfitting (high bias). To overcome bagging's shortcomings, let's discuss another technique called boosting.

Boosting

In ML, boosting involves training several weak classifiers sequentially to reduce prediction errors. The phrase "weak classifier" refers to a simple classifier that performs slightly better than random guesses. In boosting, multiple weak classifiers are converted into a single strong learning model. Figure 116 shows an ensemble of three weak classifiers.

Figure 116: A boosting example

Pros:

- **Boosting reduces bias and variance**. Combining weak classifiers leads to a strong model that's less sensitive to the changes in data. Boosting is a powerful technique to fight underfitting.
- **Slower training and inference**. Given the classifiers are trained based on the mistakes of the previous classifiers, they work sequentially. This adds to the serving time due to the sequential nature of boosting.

The boosting method is usually preferred over bagging in practice because bagging is not helpful in cases of bias, whereas boosting reduces the effect of both bias and variance.

Typical boosting-based decision trees are Adaboost [74], XGBoost [75], and Gradient boost [76]. They are commonly employed in non-classification models.

GBDT

GBDT is a commonly used tree-based ensemble model. It combines the boosting decision trees. Some variants of GBDT, such as XGBoost [75], have demonstrated strong performance in various ML competitions [77]. If you're interested in learning more about GBDT, refer to [78].

Figure 116: A GBDT model with a binary output

Here are the pros and cons of the GBDT model:

Pros:

- **Easy data preparation**. Similar to decision trees, it does not require data preparation.
- **Fast training and inference**. Compared to other boosting models.
- **Reduces bias**. GBDT reduces the prediction error by leveraging several weak classifiers, iteratively improving upon the misclassified data points from the previous classifiers.
- Works well with structured data.

Cons:

- **Lack of hyperparameters to tune**, such as the number of iterations, tree depth, regularization parameters, etc.
- GBDT does not work well on unstructured data such as images, videos, audio, etc.
- **Unsuitable for continual learning** from streaming data.

In our case, since the created features are structured data, GBDT or one of its variants such as XGBoost - is a good choice to experiment with.

A major drawback of GBDT is that it is unsuitable for continual learning. In an event recommendation system, new data continuously becomes available to the system, such as recent user interactions, registrations, new events, and even new users. In addition, users' tastes and interests may change over time. It is a good event recommendation system to adapt itself to new data, continuously. Without the possibility of continual learning, it is very costly to retrain GBDT from scratch regularly. Next, we explore neural networks which overcome this limitation.

Neural network (NN)

In an event recommendation system, we have many features that might not correlate linearly with the outcome. Learning these complex relationships is difficult. In addition, continual learning is necessary to adapting the model to new data.

NNs are great at solving these challenges. They are capable of learning complex tasks with non-linear decision boundaries. Additionally, NN models can be fine-tuned on new data using newly, making them ideal for continual learning. If you are unfamiliar with the details of NNs, you are encouraged to read [79].

Figure 117: Neural network-based model

Let's see its pros and cons.

Pros:

- **Continual learning**: NNs are designed to learn from data and improve themselves continually.
- Works well with unstructured data such as text, image, video, or audio.
- **Representation**: Neural networks have expressive power due to a fairly high number of learning parameters. They can learn very complex tasks and train these decision boundaries.

Cons:

- **Computationally expensive** to train.
- **Lack of inherent interpretability**. **Interpretation the outcomes**: NNs are sensitive to input data. For example, if input features are on a very different scale, the model may converge slowly during the training phase. An important step for NNs is data preparation, such as normalization, log-scaling, one-hot encoding, and so on.
- **Large training data** is required to train NNs.
- **Black box nature**: NNs are not interpretable, meaning it's not easy to understand the influence of each feature upon the outcome, as the model learns all through multiple layers of non-linear transformations.

Which model should we select?

Picking the right model is challenging. We often need to experiment with different models to determine which works best. We can choose the right model based on various factors:

- Complexity of the task
- Data distribution and data type
- Product requirements or constraints, such as training cost, speed, model size, etc. In this problem, both GBDTs and NNs are good candidates for experimentation. We start with the GBDT variant, XGBoost, since it is fast to implement and train. The result can be used as an initial baseline.

Once we have a baseline, we explore the possibilities of building a better model with NNs. Neural networks are expected to work well here for the following reasons:

- Massive training data is available in our system. Users continuously interact with the system by registering for events, inviting friends, publishing new events, etc. Given the number of users, this creates a significant amount of data available for training.
- Data may not be linearly separable, and neural networks can learn more complex data.

When designing a NN architecture, several hyperparameters need to be considered, including the number of hidden layers, neurons in each layer, activation functions, etc. These can be determined by employing hyperparameter tuning techniques. We architected our model, inspired by the main focus of IML system design interviews, since there is no systematic way to choose the right architecture.

Model training

Constructing the dataset

Building training and evaluation datasets is an essential step in developing a model. For example, let's look at how we compute features and their labels.

To construct a single data point, we extract a '.user.event' pair from the interaction data and compute the input features from the user. We then label the dataset with 1 if the user has registered for the event, and 0 if not.

	Extracted (user, event) features				Label
1	5	0	1	0	1
2	3	1	0	1	0

Figure 118: Constructed dataset

One issue we may face after constructing the dataset is class imbalance. The reason is that users may explore tens or hundreds of events before registering for one. Therefore, the number of negative (.user, event .) pairs is significantly higher than positive data points. We can use one of the following techniques to address the class imbalance issue:

- Use focal loss or class-balanced loss to train the classifier
- Undersample the majority class

Choosing the loss function

Since the model is a binary classification model, we use a typical classification loss function such as binary cross-entropy to optimize the neural network model.

Figure 119: Loss between the prediction and the label

Evaluation

Offline metrics

To evaluate the ranking system, we consider the following metrics:

Recall@k or Precision@k. These metrics are not good fits because they do not consider the ranking quality of the output.

MR, mAP@k, or mAR. These ratios metrics are commonly used to measure ranking quality. But which one is best?

MR focuses on the rank of the first relevant item in the list, which is suitable in scenarios where only one relevant item is expected to be retrieved. However, in an event recommendation system, several recommended events may be relevant to the user. MR is not a good fit.

mAP@k ranks well when the relevance score between a given item and a user is binary. In contrast, mAP assigns only when the item is relevant to the user. Since events are either relevant (a user registered for it) or irrelevant (a user saw the event but did not register), mAP is a better fit.

Online metrics

In our case, the business objective is to increase revenue by increasing ticket sales. To measure the impact of the system on revenue, let's explore the following metrics:

- Click through rate (CTR)
- Conversion rate
- Bookmark rate
- Revenue lift

CTR. A ratio showing how often users who saw recommended events go on to click on an event.

$$\text{CTR} = \frac{\text{Total number of clicked events}}{\text{Total number of recommendations}}$$

A high CTR shows our system is good at recommending events that users click on. Having more clicks generally means more event registrations.

However, relying only on CTR as the online metric may be insufficient. Some events are clickbait. Ideally, we would like to measure how relevant recommended events are for the user. This metric is called the conversion rate, which we discuss now.

Conversion rate. A ratio showing how often users who saw recommended events go on to register for them. The formula is:

$$\text{Conversion rate} = \frac{\text{Total number of event registrations}}{\text{Total number of clicks}}$$

A high conversion rate indicates users register for recommended events more often. For example, a conversion rate of 0.8 means that users, on average, register for 3 events out of every 10 recommended events.

Bookmark rate. A ratio showing how often users bookmark recommended events. This is based on the assumption that the platform allows users to save or bookmark an event.

Revenue lift. This is the increase in revenue as a result of event recommendations.

Serving

In this section, we propose an ML system design that can be used to serve requests. As Figure 120 shows, there are two main pipelines in the design:

- Online learning pipeline
- Prediction pipeline

Figure 120: ML system design

Online learning pipeline

As described earlier, event recommendations are intrinsically cold-start and suffer from a constant new-item problem. Consequently, the model must be continuously fine-tuned to adapt to new data. This pipeline is responsible for continuously training new models by incorporating new data, evaluating the trained models, and deploying them.

Prediction pipeline

The prediction pipeline is responsible for predicting the top-k most relevant events to a given user. Let's discuss some of the most important components of the prediction pipeline.

Event filtering

The event filtering component takes the query user as input and narrows down the events from 1 million to a small subset of events. This is based upon simple rules, such as event locations, or other types of user filters. For example, if a user adds a "concerts only" filter, the component quickly narrows down the list of available events. Since these types of filters are a common in event recommendation systems, they can be cached to significantly reduce our search space from potentially millions of events, to hundreds of candidate events.

Figure 121: Event filtering input/output

Ranking service

This service takes the user and candidate events produced by the filtering component as input, computes features for each .user, event . pair, sorts the events based on the probabilities predicted by the model, and outputs a ranked list of top k most relevant events to the user.

Figure 122: Ranking service workflow

Ranking service interacts with the feature computation component responsible for computing features that the model expects. Static features are obtained from a feature store, while dynamic features are computed in real-time from the raw data.

Other Talking Points

If there is extra time at the end of the interview, here are some additional talking points:

- What are the different types of bias we may observe in this system [71]
- How to ensure the system is fair to different user groups [72]
- Users want the ML to be a black box in most cases. How to ensure the recommended events are diverse and fresh [101]
- Describe the user's interaction with the system. We also only saw users' five locations. What are additional considerations related to privacy and security [102]
- Event management platforms are usually two-sided marketplaces, where event hosts are the suppliers and users fulfill the demand side. How to ensure enough supply to meet the demand, and vice-versa? Additionally, how to keep the platform fair for different hosts? To learn more about unique challenges in two-sided marketplaces, refer to [25].
- Fraud is a big problem for event platforms. How to detect it [103]?
- How to determine the right frequency to update the models [21]

References

1. Learning to rank interfaces, https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/tr-2005-135.pdf
2. Facebook Lasers, https://scontent.xx.fbcdn.net/v/t39.8562-6/240836140_1013529699389967_7253356109_n.pdf?_nc_cat=0&ccb=1-7&_nc_sid=ae5e01&_nc_ohc=Ob4ZIqB7qMQAX8ZKLYT&_nc_ht=scontent.xx.fbcdn.net&oh=00_AfCqqqqqqqqqqqqqqqqqqqqqqqqq&oe=6F0A0A0A
3. LambdaRank paper, https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/MSR-TR-2010-82.pdf
4. LambdaMART paper, https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/MSR-TR-2010-82.pdf
5. Surfface paper, https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/MSR-TR-2010-82.pdf
6. Learning to Rank Recommendations with Multi-Period Reinforcement Learning, https://arxiv.org/pdf/2302.01580.pdf
7. LinkedIn paper, https://dl.acm.org/doi/10.1145/2792838.2796550
8. Batch processing in stream processing, https://www.confluent.io/blog/stateful-vs-real-time-data-processing/#.~:text=Batch%20processing%20is%20when%20the,data%20is%20collected%20and%20processed
9. Leveraging location data in ML systems, https://eugeneyan.com/writings/leveraging-geolocation-data-for-machine-learning-systems/
10. https://arxiv.org/pdf/1810.09591.pdf
11. https://www.youtube.com/watch?v=0Ktzt7Ln3ec&t=1s&ab_channel=TechLeadership
12. Logistic regression, https://www.youtube.com/watch?v=ntWWaqqb8
13. Decision tree, https://scikit-learn.org/stable/modules/tree.html#tree-algorithms-id3-c4-5-c5-0-and-cart
14. Random forests, https://scikit-learn.org/stable/modules/ensemble.html#forest
15. Gradient boosting machines, https://www.youtube.com/watch?v=3CC4N4z-86I&t=1s&ab_channel=StatQuest
16. AdaBoost, https://scikit-learn.org/stable/modules/ensemble.html#adaboost
17. XGBoost, https://xgboost.readthedocs.io/en/latest/
18. Gradient boosting, https://developers.google.com/machine-learning/crash-course/ensembles/gradient-boosted-decision-trees
19. XGBoost in Kaggle competitions, https://www.kaggle.com/search?q%5BXGBoost%5D
20. Wide and Deep learning, https://www.tensorflow.org/tutorials/wide_and_deep
21. An introduction to GBDT, https://www.machinelearningplus.com/machine-learning/an-introduction-to-gradient-boosting-decision-tree/
22. Introduction to neural networks, https://www.youtube.com/watch?v=ORHwaqbllxs
23. Bias issues and solutions in recommender system, https://www.youtube.com/watch?v=xzPnzDcZ22k
24. Recommendation system at Amazon, https://www.youtube.com/watch?v=RfJ0Ldt0qcY&ab_channel=ACMRecSys
25. Challenges and diversity in recommendation systems, https://developers.google.com/machine-learning/recommendation/dnn/softmax-training
26. Privacy and security in ML, https://www.microsoft.com/en-us/research/blog/privacy-preserving-machine-learning-maintaining-confidentiality-and-preventing-attacks/
27. Two-sided marketplace challenges, https://www.aleo.com/blog/two-side-recommendation-marketplace/
28. Data leakage, https://machinelearningmastery.com/data-leakage-machine-learning/
29. Online training frequency, https://huyenchip.com/2022/01/02/real-time-machine-learning-challenges-and-solutions.html#towards-continual-learning

Back to Complete

Partner With Us
Be an Instructor
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
Contact Us
Report a Bug

Resources
YouTube
Visual Site Guide
Find Jobs
Prepare for Coding Interviews

Copyright ©2020-2026 BytByteGo Inc. All rights reserved.